

BZ 214 Visual Programming

Robot Vacuum Cleaning Simulation

1. Overview

Robot vacuum cleaners are commonly used in modern homes to automatically clean floors. However, real robot vacuum systems must deal with different types of dirt, furniture obstacles, limited battery capacity, and the need to return to a charging station.

This project asks you to create a Robot Vacuum Cleaning Simulation in which a robot vacuum moves through a room, avoids obstacles, detects and cleans different types of dirt, manages its battery, and returns to the charging station when necessary.

You will develop the application in Java, using JavaFX for the graphical interface. Make sure to use object-oriented programming principles and follow the Model-View-Controller (MVC) design pattern. A representative visual of the proposed Robot Vacuum Cleaning Simulation, generated with the help of ChatGPT, is shown in Figure 1.

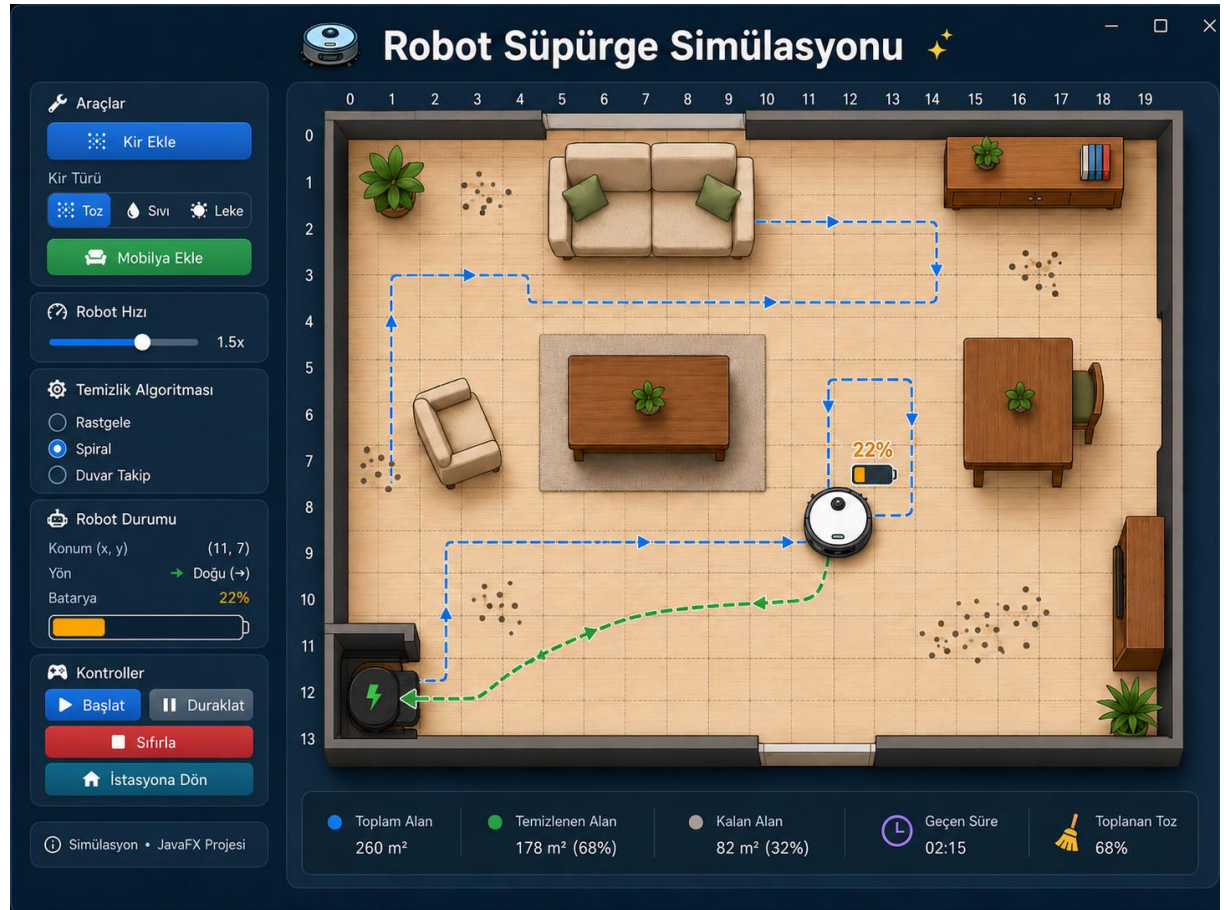


Figure 1. A representative user interface of the Robot Vacuum Cleaning Simulation generated with the help of ChatGPT

2. Functional Requirements

The system must support the following features:

- The graphical interface must display the room grid structure, robot vacuum cleaner, obstacles such as furniture and tables, dirt, charging station, and the robot's movement path.
- The user interface must include options such as adding dirt, selecting the dirt type, adding obstacles, changing the robot speed, and selecting the cleaning algorithm.
- Through the user interface, the user must be able to start, pause, and reset the simulation, send the robot back to the charging station, and manually update the robot's battery level.
- The system must support at least three different dirt types: dust, liquid, and stain.
- The robot must have the following behaviors:
 - The robot must not pass through walls or furniture. Changing direction when it hits a wall or furniture
 - Cleaning a dirty cell when it passes over it
 - Decreasing the battery while moving
 - Consuming additional battery, depending on the dirt type
 - Returning to the charging station when the battery is low
- During the simulation, information such as the robot's position, direction, battery level, cleaned area percentage, remaining dirty area, and elapsed time must be updated in real time.
- The robot must return to the charging station using the shortest possible path (For example using BFS or A* path-finding) when the battery level is low or when the user selects the "Return to Charging Station" option.
- The system must clean different dirt types in different durations. For example, dust should be cleaned more quickly, while liquids and stains should require more cleaning time.

3. Technical Specifications

- Use JavaFX as the GUI library.
- Strict adherence to the MVC architecture.
- Follow best practices in Object-Oriented Programming.
- Use Java Collection Frameworks (e.g., List, Map, Queue) as appropriate.
- No third-party libraries may be used—only standard Java SE libraries are permitted. (If you really need to use a third-party library, please ask for permission first.)

4. Optional Bonus Features

You can earn bonus consideration by implementing:

- Unreachable area detection
- Multiple room layouts
- Sound effects
- Animation like cleaning

5. Documentation

At the end of your project, you must prepare a detailed report that describes your work. You must provide details about your project, specify the class, and include use case diagrams.

6. Academic Honesty

Honesty and integrity are essential. Students must not submit someone else's code, design, or report as their own. Copying code from another student, an online source, or an AI-generated project without understanding and modifying it is not allowed. A student who copies the code from another source or allows another student to copy their work may receive 0 points.

7. Submission Checklist

Please submit a ZIP archive named with your group number. For example, "group1.zip".

The ZIP file must contain:

- All source code files .java
- JavaFX project files
- Image or asset files used in the project
- README or usage manual
- Project report
- Screenshots of the running application
- Optional simulation video or animated GIF

8. Public Sharing and Acknowledgment

If the project is shared publicly on any online platform, such as GitHub, GitLab, Bitbucket, a personal website, or a portfolio page, the course name must be clearly mentioned in the project description or README file. Students are expected to acknowledge that the project was developed as part of the **BZ 214 Visual Programming** course.

Suggested acknowledgment text:

"This project was developed as part of the BZ 214 Visual Programming course. Special thanks to the course instructor and contributors."

If the project idea, requirements, or assignment document is used or referenced publicly, proper acknowledgment must be given.

If you have any questions or concerns about the submission process, please don't hesitate to ask.

*Special thanks to **Gökhan AZİZOĞLU**.*